

# Reporte Tecnico - Proyecto Final OpenMP

Materia: Programacion Paralela y Concurrente - D04

Alumno: HUGO JOHNATHAN JAUREGUI RABELERO

Fecha: 25 de mayo de 2026

## 1. Resumen del proyecto

Este proyecto implementa una aplicacion en C++ que genera una imagen del conjunto de Mandelbrot y despues aplica un filtro de convolucion Gaussiano pesado. Se incluye una version secuencial, una linea base paralela con OpenMP generada por IA y una version optimizada manualmente con comparacion de schedulers, histograma, vectorizacion SPMD y pruebas de afinidad de hilos para obtener los 10 puntos extra.

## 2. Repositorio de codigo

Enlace de GitHub: \_\_\_\_\_

El historial de commits debe quedar asi:

- Commit 1: codigo secuencial base.
- Commit 2: linea base paralela generada por IA con OpenMP.
- Commit 3: optimizaciones manuales de OpenMP, schedulers, histograma, SPMD y afinidad.
- Commit 4: reporte tecnico, resultados y graficas.

## 3. Especificaciones del hardware de prueba

Ejecutar: `./scripts/collect_hardware.sh` y copiar aqui los datos principales desde `results/hardware_info.txt`.

| Elemento        | Dato                                     |
|-----------------|------------------------------------------|
| Modelo de CPU   | Completar con <code>lscpu/free -h</code> |
| Nucleos fisicos | Completar con <code>lscpu/free -h</code> |
| Nucleos logicos | Completar con <code>lscpu/free -h</code> |
| Cache L1        | Completar con <code>lscpu/free -h</code> |
| Cache L2        | Completar con <code>lscpu/free -h</code> |
| Cache L3        | Completar con <code>lscpu/free -h</code> |
| RAM             | Completar con <code>lscpu/free -h</code> |

## 4. Prompts exactos usados con IA

Prompt para codigo secuencial base:

Genera un programa en C++ puramente secuencial que cree una imagen de ultra alta resolucion del conjunto de Mandelbrot y despues aplique un filtro de convolucion 2D pesado, preferentemente un desenfoque Gaussiano de radio amplio. El programa debe medir por

separado el tiempo de generacion del fractal y el tiempo del filtro, guardar la imagen en formato PPM y usar estructuras simples para poder paralelizarlo despues con OpenMP.

Prompt para linea base paralela de IA:

Paraleliza el codigo secuencial anterior usando OpenMP. Usa `#pragma omp parallel for` en los bucles principales de Mandelbrot y del filtro de convolucion. Conserva la misma salida y agrega medicion de tiempos para comparar contra la version secuencial.

## 5. Metodologia

1. Primero se compilo y ejecuto la version secuencial para obtener el tiempo base.
2. Despues se compilo la linea base paralela de IA usando OpenMP con `schedule(static)`.
3. Luego se implemento una version optimizada con `schedule(runtime)`, permitiendo probar `static`, `dynamic` y `guided` con distintos `chunk size`.
4. Se midio el rendimiento desde 1 hilo hasta el doble de nucleos logicos.
5. Se agrego un histograma de intensidad de la imagen final con dos enfoques: `atomic` y contadores locales por hilo.
6. Se agrego `#pragma omp simd` en los bucles internos del filtro para forzar vectorizacion SPMD.
7. Para los 10 puntos extra se midio con `OMP_PROC_BIND=false`, `close` y `spread` usando `OMP_PLACES=cores`.

## 6. Errores logicos o cuellos de botella detectados en el codigo inicial de IA

- La linea base paralela usa `schedule(static)`, que no siempre balancea bien Mandelbrot porque algunas regiones tienen mas iteraciones que otras.
- El histograma con `atomic` genera mucha contencion cuando muchos hilos incrementan los mismos contadores.
- Guardar contadores compartidos sin padding puede provocar false sharing porque varios hilos escriben en datos cercanos dentro de la misma linea de cache.
- El filtro de convolucion consume mucha memoria y puede quedar limitado por ancho de banda/cache, por eso la afinidad de hilos puede influir.

## 7. Resultados esperados y archivos generados

- `results/benchmark.csv`: tiempos por numero de hilos.
- `results/scheduler.csv`: comparacion de `static`, `dynamic` y `guided` con `chunk size`.
- `results/tiempo_vs_hilos_optimized.png`: grafica de tiempo de ejecucion vs hilos.
- `results/speedup_vs_hilos_optimized.png`: grafica de aceleracion vs hilos.
- `results/schedulers_taskA.png`: comparacion de schedulers para Tarea A.
- `results/vectorization_report.txt`: evidencia de vectorizacion del compilador.
- `results/histogram_local.csv` y `results/histogram_atomic.csv`: histogramas calculados.

## 8. Grafica de tiempo de ejecucion vs numero de hilos

Insertar aqui la grafica generada por scripts/plot\_results.py:  
results/tiempo\_vs\_hilos\_optimized.png

## 9. Grafica de speedup vs numero de hilos

Insertar aqui la grafica generada por scripts/plot\_results.py:  
results/speedup\_vs\_hilos\_optimized.png

## 10. Analisis de schedulers

Con base en results/scheduler.csv, el scheduler optimo para mi procesador fue:  
\_\_\_\_\_ con chunk size: \_\_\_\_\_.

Justificacion: Se selecciona el que obtiene el menor tiempo en Tarea A. En Mandelbrot la carga no es uniforme, por lo que dynamic o guided pueden mejorar el balanceo frente a static, aunque con chunks demasiado pequenos aumenta el overhead.

## 11. Sincronizacion, histograma y false sharing

La implementacion con atomic es correcta pero puede ser lenta por contencion. La implementacion con variables locales por hilo reduce la sincronizacion y despues combina los resultados. Tambien se uso alignas(64) para separar histogramas locales por linea de cache y reducir false sharing.

Resultado observado: hist\_atomic = \_\_\_\_\_ s, hist\_local = \_\_\_\_\_ s. La version local fue \_\_\_\_\_ veces mas rapida.

## 12. SPMD y vectorizacion

Se uso #pragma omp simd reduction(+:r,g,b) en los bucles internos del filtro Gaussiano. La evidencia se encuentra en results/vectorization\_report.txt. Pegar aqui una linea del reporte donde el compilador confirme loop vectorized:

\_\_\_\_\_

## 13. Extra: afinidad OMP\_PROC\_BIND y OMP\_PLACES

Se ejecutaron pruebas con OMP\_PLACES=cores y OMP\_PROC\_BIND=false, close y spread. Comparar los tiempos en results/benchmark.csv.

| Configuracion  | Tiempo total | Mejora vs sin afinidad | Comentario |
|----------------|--------------|------------------------|------------|
| false + cores  |              |                        |            |
| close + cores  |              |                        |            |
| spread + cores |              |                        |            |

## 14. Analisis de rendimiento: Ley de Amdahl y overhead

La Ley de Amdahl limita la aceleracion porque no todo el programa es paralelizable y porque existen costos de sincronizacion, memoria y administracion de hilos. El punto donde el rendimiento deja de mejorar se identifica en la grafica de speedup: a partir de \_\_\_\_\_ hilos el speedup se estanca o baja. Esto indica que el overhead del sistema operativo, la contencion de memoria/cache o la sobre-suscripcion de hilos supera el beneficio de agregar mas paralelismo.

## 15. Conclusiones

- OpenMP permitio reducir el tiempo de ejecucion respecto a la version secuencial.
- El scheduler optimo debe elegirse empiricamente porque depende del procesador y del patron de carga.
- En Mandelbrot, dynamic/guided pueden balancear mejor que static cuando hay zonas con distinta cantidad de iteraciones.
- El histograma local por hilo evita gran parte de la contencion de atomic y reduce problemas de false sharing.
- La vectorizacion SPMD ayuda al filtro de convolucion, pero su rendimiento tambien depende de cache y ancho de banda de memoria.
- La afinidad de hilos puede mejorar o empeorar segun la arquitectura, por eso se midio con OMP\_PROC\_BIND y OMP\_PLACES.

## 16. Instrucciones de reproduccion

Comandos usados:

- `sudo apt update`
- `sudo apt install -y build-essential g++ make git python3 python3-pip`
- `pip3 install pandas matplotlib`
- `./scripts/collect_hardware.sh`
- `make clean && make all`
- `W=3840 H=2160 ITER=1000 RADIUS=15 ./scripts/run_benchmarks.sh`
- `python3 scripts/plot_results.py`